



Universitat
de les Illes Balears

PRÀCTICA:

21780 - COMPILADORS

Jose Maria Macias

45613303W

Biel Perelló

41657594V

ÍNDEX

Objectiu.....	3
Funcionalitat del Processador.....	4
Front-end.....	4
Back-end.....	4
Gestió d'errors.....	5
Característiques del llenguatge.....	5
Estructura del programa.....	5
Tipus de dades.....	5
Declaracions i valors.....	6
Operacions i estructures de control.....	6
Expressions.....	6
Entrada i sortida.....	6
Operadors disponibles.....	7
Especificació del llenguatge.....	8
Descripció del llenguatge.....	8
Estructura general del programa.....	8
Tipus de dades.....	10
Operacions.....	11
Funcions i procediments.....	14
Entrades i sortides.....	15
Gramàtica.....	16
Disseny del compilador.....	18
Anàlisi lèxica Lexer.flex / AnaLex.....	18
Anàlisi sintàctica — Parser.cup.....	20
Anàlisi semàntica — SemanticAnalyzer.java.....	22
Gestió de la taula de símbols.....	25
Gestió d'errors.....	26
Gestió de memòria.....	27
Generació de codi.....	30
Optimitzacions.....	30
Generació de codi assemblador.....	31
Execució i validació.....	34
Llibreries necessàries.....	34
Procediment d'execució.....	34
Casos de prova.....	36
Cas de prova 1.....	37
Cas de prova 2.....	38
Cas de prova 3.....	39
Cas de prova erroni 1.....	40
Cas de prova erroni 2.....	41
Cas de prova erroni 3.....	43
Conclusions.....	44

Objectiu

L'objectiu principal d'aquesta pràctica és **dissenyar i implementar un compilador complet per a un llenguatge imperatiu**, desenvolupat específicament per al projecte. El treball comprèn totes les fases clàssiques del procés de compilació, des de l'anàlisi del codi font fins a la generació de codi ensamblador, passant per la validació semàntica i la producció de representacions intermèdies.

Per assolir aquest objectiu, el compilador ha de ser capaç de:

1. **Processar programes escrits en el llenguatge definit a l'enunciat..**
2. **Realitzar correctament totes les fases del front-end:**
 - **Anàlisi lèxica**, identificant i classificant tots els tokens del programa.
 - **Anàlisi sintàctica**, comprovat que la seqüència de tokens s'ajusta a la gramàtica definida del llenguatge.
 - **Anàlisi semàntica**, validant el tipus i ús de les expressions, la coherència de declaracions, crides a procediments i verificació de regles contextuais.
3. **Proporcionar mecanismes efectius de detecció i gestió d'errors**, generant informes diferenciats per errors lèxics, sintàctics i semàntics, amb informació precisa de línia i naturalesa de l'error.
4. **Implementar el back-end complet**, incloent:
 - **Generació de codi intermedi** basat en instruccions de tres adreces, mantenint l'estructura semàntica del programa original.
 - **Optimitzacions bàsiques**, millorar l'eficiència del codi intermedi quan s'escau.
 - **Generació de codi ensamblador**, tant en versió directa com optimitzada, assegurant que l'executable resulti funcionalment equivalent en ambdós casos.
5. **Generar tots els fitxers de sortida requerits**, incloent:
 - Seqüència de tokens.
 - Taula de símbols final.
 - Taules de variables i procediments.
 - Codi intermedi.
 - Codi ensamblador sense optimitzar i optimitzat.
 - Informe d'errors, si n'hi ha.

En conjunt, aquest projecte pretén que l'estudiant apliqui de manera pràctica els conceptes fonamentals de la teoria de compiladors, entengui les interdependències entre les diferents fases del procés, i sigui capaç de construir una eina funcional, modular i extensible. L'objectiu final és obtenir un compilador completament operatiu capaç de transformar programes escrits en el llenguatge dissenyat en codi executable equivalent en assembleador, validat mitjançant casos de prova correctes i erronis

Funcionalitat del Processador

El processador implementat en aquest projecte és l'encarregat d'executar totes les fases del cicle de compilació i de generar els fitxers de sortida corresponents. El compilador ha de ser capaç de **processar qualsevol codi font subministrat en un arxiu de text**, inicialitzant totes les estructures internes (taula de símbols, generador d'etiquetes, gestió d'àmbits i control d'errors) i executant seqüencialment les fases d'anàlisi i traducció.

Pel que fa a la seva funcionalitat, el processador genera els elements següents:

Front-end

- **Fitxer de tokens**, que recull tots els tokens identificats per l'analitzador lèxic en l'ordre d'aparició.
- **Taula de símbols final**, construïda durant l'anàlisi sintàctica i semàntica, que conté informació completa sobre identificadors, tipus, àmbits, paràmetres i procediments.

Back-end

- **Taules de variables i procediments**, utilitzades per verificar la coherència del codi intermedi generat.
- **Fitxer de codi intermedi**, que conté la representació en tres adreces del programa, resultat de l'execució de les regles semàntiques.
- **Fitxer de codi assembleador no optimitzat**, on cada instrucció de tres adreces apareix comentada amb la seva traducció directa.
- **Fitxer de codi assembleador optimitzat**, equivalent funcionalment però amb millores de rendiment derivades de l'eliminació de redundàncies o simplificacions.

Gestió d'errors

Si durant alguna fase del procés es detecten errors lèxics, sintàctics o semàntics, el processador genera un **document d'errors** que inclou, per a cada cas:

- la línia on s'ha produït,
- el tipus d'error,
- un missatge descriptiu.

Aquesta funcionalitat permet que el compilador actuï com una eina de diagnòstic completa, facilitant la depuració del programa font i garantint la traçabilitat de totes les fases del processament.

Característiques del llenguatge

El llenguatge dissenyat per a aquest projecte és un llenguatge imperatiu amb una estructura clara de programa i un conjunt acotat de tipus, operacions i construccions de control. Les característiques implementades són les següents:

Estructura del programa

- Cos general del programa que conté declaracions, definició de subprogrames i el conjunt d'instruccions principals, d'una manera similar al concepte de *main* en llenguatges com Java o C++, o a un bloc d'instruccions com en Python.
- Definició de subprogrames mitjançant **funcions** i **procediments**, amb suport per a arguments.

Tipus de dades

El llenguatge incorpora els tipus primitius següents:

- Enter
- Caràcter
- Cadena de caràcters
- Lògic

A més, s'admet un tipus definit per l'usuari:

- **Tuples**, amb un nombre i tipus de camps determinats.

Declaracions i valors

- Declaració i ús de variables de qualsevol dels tipus contemplats.
- Declaració de constants.
- Possibilitat d'utilitzar literals i valors associats als tipus definits.

Operacions i estructures de control

El llenguatge permet les operacions següents:

- Assignació.
- Condicionals (*if-else*).
- Selecció múltiple tipus *switch* (si s'utilitza).
- Estructures de repetició:
 - **while**
 - **do-while**
- Crida a procediments i funcions amb paràmetres.
- Retorn de valors en funcions.

Expressions

Les expressions del llenguatge poden estar formades per:

- Literals del tipus corresponent.
- Constants i variables.
- Operacions aritmètiques, relacionals i lògiques.

Entrada i sortida

El llenguatge incorpora operacions bàsiques d'I/O:

- Entrada per teclat.
- Sortida per pantalla.

Operadors disponibles

- **Aritmètics:** suma, resta, producte, divisió.
- **Relacionals:** igual, diferent, major, menor, major o igual, menor o igual.
- **Lògics:** i, o i no.

Especificació del llenguatge

Descripció del llenguatge

El llenguatge de programació dissenyat per a aquesta pràctica és un llenguatge imperatiu d'alt nivell que utilitza el català com a llengua per a totes les paraules clau i construccions sintàctiques. L'objectiu principal del llenguatge és proporcionar una sintaxi clara, estructurada i accessible, que permeti expressar programes de manera senzilla tot mantenint les capacitats essencials dels llenguatges moderns.

Aquest llenguatge ofereix mecanismes per a la declaració i ús de variables i constants, la definició de funcions i procediments amb paràmetres, i l'expressió d'operacions bàsiques sobre dades primitives i estructurades. Incorpora estructures de control com condicions i bucles, així com operacions d'entrada i sortida simples, fet que permet redactar programes complets orientats al càlcul i al flux de control determinista.

A més, el llenguatge admet tipus compostos en forma de tuples, oferint la possibilitat d'agrupar conjuntament diversos valors sota una mateixa entitat. La combinació d'aquests elements proporciona un entorn expressiu suficient per implementar lògiques de programació representatives, mantenint alhora un disseny compacte i pedagògic, adequat per a l'objectiu formatiu de la pràctica i per al procés de construcció del compilador.

Estructura general del programa

Un programa en aquest llenguatge s'organitza en un únic bloc principal, declarat amb la paraula clau `principal`, que conté tant les declaracions de subprogrames i variables com les instruccions que s'han d'executar. La finalització del bloc principal es marca amb la paraula clau `fprincipal`. La forma general és la següent:

```
principal nom:
  ## Declaracions de variables
  ## Definicions de procediments i funcions
  inici
    ## Instruccions del programa
  fi
fprincipal
```

Components de l'estructura

Capçalera del programa

Un programa comença amb:

```
principal ID:
```

on **ID** representa el nom del programa.

Declaracions

Abans del bloc d'instruccions principals, el llenguatge permet incloure:

- Declaracions de variables (**decl**)
- Definicions de procediments o funcions (**method**)

Aquestes declaracions poden definir variables globals del programa o bé subprogrames que seran utilitzats posteriorment.

Bloc principal d'instruccions

El cos executable del programa s'inicia amb la paraula clau:

```
inici
```

i finalitza amb:

```
fi
```

Aquest bloc conté totes les instruccions que el programa executa en seqüència.

Instruccions del llenguatge

Dins del bloc principal —i també dins subprogrames— poden aparèixer instruccions de diferents tipus, com referències, condicions, bucles, entrada/sortida, assignacions o declaracions locals.

A nivell sintàctic, el llenguatge permet combinar lliurement aquests elements per formar el flux d'execució.

Comentaris

Els comentaris utilitzen el prefix:

```
##
```

Qualsevol text des de `##` fins al final de línia és ignorat pel compilador. Aquests comentaris no generen cap token ni intervenen en la semàntica del programa.

Aquesta estructura general proporciona un esquema clar i regular per a la construcció de programes, separant explícitament la zona de declaracions i la zona d'execució, i mantenint una sintaxi simple basada en paraules clau en català.

Tipus de dades

El llenguatge incorpora un conjunt reduït però suficient de tipus de dades, que inclou tant tipus primitius com un tipus compost definit per l'usuari. Aquest disseny permet representar informació bàsica i estructurada mantenint una sintaxi simple.

Tipus bàsics

Els tipus primitius disponibles són els següents:

- **Enter**
 - Representa nombres enters.

```
enter variable_entera := 5
```

- **Caracter**
 - Representa un únic caràcter, delimitat per cometes simples.

```
caracter lletra := 'A'
```

- **Cadena**
 - Representa una seqüència de caràcters, delimitada per cometes dobles.

```
cadena missatge := "Hola món"
```

- **Lògic**
 - Representa un valor booleà amb dos literals possibles: `cert` i `fals`.

```
logic estat := cert
```

Tipus compostos

El llenguatge ofereix un únic tipus compost:

- **Tupla**
 - Permet agrupar diversos camps heterogenis sota una mateixa entitat. La definició d'una tupla segueix el format:

```
tupla {camp1: tipus1, camp2: tipus2, ... } NomTupla
```

L'accés als camps d'una tupla es realitza mitjançant la notació amb punt:

```
NomTupla p  
enter edat_persona := p.edat  
p.edat := 20
```

Declaració de variables i constants

Les variables poden declarar-se utilitzant qualsevol dels tipus definits. Les constants es declaren mitjançant la paraula clau **const**, després de la qual el valor queda fixat durant tota l'execució del programa.

```
enter x := 10;  
const logic mode_debug := fals  
cadena salutacio := "Bon dia"
```

Operacions

El llenguatge proporciona un conjunt d'operacions bàsiques que permeten expressar assignacions, estructures de control, selecció múltiple, iteracions i combinacions lògiques i aritmètiques. La sintaxi utilitza paraules clau en català i operadors definits explícitament en l'especificació lèxica.

Assignació

L'assignació es realitza mitjançant l'operador **:=**, seguint l'esquema:

```
variable := expressio
```

On **expressio** pot ser un literal, una variable, una operació, una crida a funció o un accés a estructura.

```
x := 10;  
missatge := "Hola"  
estat := cert
```

Estructures condicionals

El llenguatge inclou l'estructura condicional clàssica **si - llavors - sino - fsi**.

1. Forma completa:

```
si (condicio) llavors  
    instruccions  
sino  
    instruccions  
fsi
```

2. Forma simplificada (sense bloc else):

```
si (condicio) llavors  
    instruccions  
fsi
```

La condició ha de ser de tipus **logic**.

Selecció múltiple

El llenguatge incorpora una estructura de selecció múltiple basada en la paraula clau **cas**, que permet seleccionar executables segons el valor d'una expressió.

```
cas variable:  
    valor1: instruccions  
    valor2: instruccions  
    ...  
    altre: instruccions  
fcas
```

La clau **altre** defineix el cas per defecte.

Bucles

El llenguatge implementa dos tipus de bucles:

1. Bucle *mentre* (**while**)

Executa repetidament un conjunt d'instruccions mentre la condició és certa:

```
mentre (condicio) fer
    instruccions
fmentre
```

2. Bucle *fer...repetir* (**do-while**)

Executa el bloc almenys una vegada i avalua la condició al final:

```
fer
    instruccions
repetir (condicio)
```

Operadors

El llenguatge suporta els operadors següents:

Aritmètics (entre enters i concatenació per cadenes)

- **+** (suma), **-** (resta), ***** (producte) i **/** (divisió)

Relacionals

- **==** (igual), **!=** (diferent), **<** (menor), **>** (major), **<=** (menor o igual) i **>=** (major o igual)

Lògics

- **i** (AND), **o** (OR) i **no** (NOT)

Aquests operadors poden aparèixer dins d'expressions aritmètiques, lògiques o condicionals.

Funcions i procediments

El llenguatge permet modularitzar el codi mitjançant la definició de **funcions** i **procediments**, que poden declarar-se abans del bloc principal del programa. Ambdós tipus de subprogrames poden rebre arguments i disposen del seu propi àmbit de variables locals.

Definició de funcions

Una funció és un subprograma que retorna un valor. La seva definició utilitza les paraules clau **funcio** i **fifuncio**, i especifica el tipus de retorn i els paràmetres formals.

```
funcio tipus nom_funcio(par1: tipus1, par2: tipus2, ...):  
    ## Declaracions de variables  
    ## Definicions de procediments i funcions  
    inici  
        ## Instruccions del programa  
    fi  
    tornar expressio  
fifuncio
```

El retorn del valor es realitza mitjançant la instrucció:

```
tornar expressio  
  
funcio enter suma(a: enter, b: enter):  
    tornar a + b  
fifuncio
```

Definició de procediments

Un procediment és un subprograma que **no retorna cap valor**, i s'utilitza per encapsular funcionalitats que produeixen un efecte però no generen un resultat directe. Es declara amb les paraules clau **procediment** i **fiprocediment**.

```
procediment nom_proc(par1: tipus1, par2: tipus2, ...):  
    ## Declaracions de variables  
    ## Definicions de procediments i funcions  
    inici  
        ## Instruccions del programa  
    fi  
fiprocediment
```

```
## Exemple de procediment
procediment escriu_msg(m: cadena):
    inici
        sortida(m)
    fi
fiprocediment
```

Crida a funcions i procediments

Les crides es fan pel nom del subprograma seguit de la llista de paràmetres:

```
nom_funcio(expressions)
nom_procediment(expressions)
x := suma(3, 5)
escriu_msg("Hola")
```

- Les **funcions** poden aparèixer dins expressions, assignacions i condicions.
- Els **procediments** s'executen com a instrucció independent, ja que no tornen cap valor.

Ambdues crides fan ús de pas de paràmetres per valor, i cada subprograma defineix el seu propi entorn d'execució i variables locals.

Entrades i sortides

El llenguatge proporciona mecanismes senzills per interactuar amb l'usuari o amb l'entorn d'execució mitjançant instruccions d'entrada i sortida. Aquestes operacions formen part del conjunt d'instruccions bàsiques i utilitzen paraules clau en català.

Sortida per pantalla

La instrucció **sortida** permet mostrar informació a la pantalla. Accepta com a argument una expressió.


```
sortida(expr1)
```

```
sortida("Escriu el teu nom:")  
sortida("Hola com va?")
```

Entrada per teclat

L'operació d'entrada utilitza la paraula clau **entrada** i permet llegir un valor introduït per l'usuari, assignant-lo a una variable. La instrucció rep com a argument la variable de destinació.

Sintaxi general:

```
entrada(variable)
```

```
cadena nom  
enter edat
```

```
entrada(nom)  
entrada(edat)
```

Gramàtica

P → principal id : SAFE_DECLS BLOC fprincipal

SAFE_DECLS → DECLS | error DECLS

DECLS → DECL DECLS | METHOD DECLS | λ

DECL → TIPUS id ASSIG | const TIPUS id := EXPR

METHOD → funcio TIPUS id (ARGS) : SAFE_DECLS BLOC tornar RET_FUNC ffuncio
| procediment id (ARGS) : SAFE_DECLS BLOC fprocediment

RET_FUNC → tornar EXPR

BLOC → inici INSTRS fi | λ

INSTRS → INSTR INSTRS | error INSTRS | λ

INSTR → REF | COND | LOOP | ENTRADA | SORTIDA | ASSIG_INSTR | DECL

ASSIG_INSTR → REF := EXPR

COND → si (EXPR) llavors INSTRS OPTIONAL_COND fsi
| cas EXPR: CAS_OPCIONS altre : INSTRS fcas

OPTIONAL_COND → sino INSTRS | λ

CAS_OPCIONS → CAS_OPCIO CAS_OPCIONS | CAS_OPCIO

CAS_OPCIO → TERME : INSTRS

LOOP → mentre (EXPR) fer INSTRS fmentre | fer INSTRS repetir (EXPR)

ENTRADA → entrada (ID)

SORTIDA → sortida (EXPR)

TIPUS → enter | cadena | caracter | logic | tupla { ARGS } | id

ARGS → ARG , ARGS | ARG | λ

ARG → id : TIPUS

ASSIG → := EXPR | λ

EXPR → EXPR_LOGIC

EXPR_LOGIC → EXPR_LOGIC OP_LOGIC EXPR_REL | EXPR_REL

EXPR_REL → EXPR_REL OP_REL EXPR_ARIT | EXPR_ARIT

EXPR_ARIT → EXPR_ARIT OP_ARIT TERME | TERME

TERME → LITERAL | REF | (EXPR) | no EXPR | - EXPR

LITERAL → valor_logic | nombre | string | caracter

REF → id | REF . id | CALL

CALL → id (EXPR_LIST)

EXPR_LIST → EXPR , EXPR_LIST | EXPR | λ

OP_ARIT → + | - | * | /

OP_REL → == | != | < | <= | > | >=

OP_LOGIC → i | o

Disseny del compilador

Anàlisi lèxica `Lexer.flex` / `AnaLex`

Aquesta pràctica implementa l'analitzador lèxic amb **JFlex**, generant la classe **`frontend.lexer.AnaLex`** a partir del fitxer **`Lexer.flex`**. L'objectiu de l'escàner és **llegir el flux de caràcters d'entrada**, agrupar-los en **lexemes** segons uns **patrons (expressions regulars)** i retornar al parser (CUP) els **tòkens** corresponents (constants de **`frontend.parser.ParserSym`**).

La configuració del generador inclou suport Unicode i manteniment de posició:

- **`%unicode`**: permet tractar caràcters Unicode a l'entrada.
- **`%cup`**: fa que l'escàner retorni objectes **`java_cup.runtime.Symbol`** compatibles amb CUP.
- **`%line`** i **`%column`**: activa el comptatge automàtic de línia i columna (0-based internament a JFlex).

Definició de patrons (macros)

El fitxer defineix un conjunt de macros (patrons reutilitzables) que encapsulen les categories lèxiques principals del llenguatge:

- **`espai = [ \t\n\r ]+`**
Captura blancs (espai, tab, salt de línia i retorn de carro). Es descarten.
- **`comentari = ##.*`**
Comentari de línia iniciat per **`##`** fins final de línia. També es descarta.
- **`id = [a-zA-Z_][a-zA-Z_0-9]*`**
Identificadors tipus llenguatges imperatius: comencen per lletra o **`_`** i poden contenir dígit després.
- **`op_aritmetic = ("+"|"-"|"/"|"*")`**
- **`op_rel = ("<="|">="|"!="|"=="|"<"|">")`**
- **`op_logic = (no|i|o)`**
- **`valor_logic = (cert|fals)`**
Literal booleà (es converteix a **`boolean`**).

- `nombre = [0-9]+(\\. [0-9]+)?([Ee][+-]?[0-9]+)?`
Literal numèric amb suport de part decimal i notació científica.
- `cadena = \"^[\\\"]*\"`
Cadena entre cometes dobles (sense cometes internes no escapades).
- `caracter = \'([\\\'\\\"|\\\\.\\\\)\\'`
Caràcter entre cometes simples, admetent escapes bàsics (p. ex. `'\\n'`, `'\\t'`, `'\\'`, `'\\\\'`).

Conjunt de tokens reconeguts

L'escàner retorna símbols `Symbol` amb el camp `sym` igual al token definit a `ParserSym`. La cobertura és:

- Assignació i delimitadors: `:=`, `:`, `,`, `;`, `.`
- Parèntesis: `( )`
- Corxets: `[ ]`
- Claus: `{ }`

Cada lexema retorna un tòken específic (p. ex. `ParserSym.ASSIGNACIO`, `ParserSym.OBR_PAR`, etc.).

- **Aritmètics:** `+`, `*`, `/` (i el `-` es tracta també com a símbol individual via regla literal `"-"`).
- **Relacionals:** `==`, `!=`, `<`, `<=`, `>`, `>=`
- **Lògics:** `i`, `o`, `no`

Es defineixen explícitament paraules clau per cadena literal i es retornen com a tòkens dedicats. Això evita que es confonguin amb `ID`. Exemples:

- Estructura principal: `principal`, `fprincipal`
- Condicionals: `si`, `llavors`, `sino`, `fsi`
- Selecció: `cas`, `altre`, `fcas`
- Iteració: `mentre`, `fer`, `fmentre`, `repetir`
- Subprogrames: `procediment`, `fiprocediment`, `funcio`, `fifuncio`, `tornar`
- Tipus: `cadena`, `caracter`, `tupla`, `enter`, `logic`

- Altres: `const`, `entrada`, `sortida`, `inici`, `fi`

Aquests tòkens duen **valor associat** (`Symbol.value`) perquè el parser i les fases posteriors puguin treballar amb informació semàntica:

- `{id}` → `ID` amb el nom (`String`).
- `{valor_logic}` → `VALOR_LOGIC` amb booleà (`true` si `cert`, `false` si `fals`).
- `{cadena}` → `CADENA` amb el contingut sense cometes.
- `{caracter}` → `CHARACTER` amb un `char` (gestionant escapes).
- `{nombre}` → `ENTER` amb valor numèric (en aquest codi es converteix via `Double.parseDouble(...)`).

Rutines i accions semàntiques

S'implementen dues rutines `symbol(...)`:

- `symbol(int type)` per tokens sense atribut.
- `symbol(int type, Object value)` per tòkens amb atribut.

Ambdues fixen:

- `left = yyline + 1`
- `right = yycolumn + 1`

Això transforma la numeració interna (0-based) a una presentació **1-based**, més adequada per a missatges d'error i informes.

Anàlisi sintàctica — Parser.cup

La part d'anàlisi sintàctic del compilador s'ha implementat mitjançant **CUP (Constructor of Useful Parsers)**, utilitzant el fitxer `Parser.cup` per definir formalment la **gramàtica del llenguatge**, les **regles de producció** i les **accions semàntiques** associades. A partir d'aquest fitxer es generen automàticament les classes `Parser` i `ParserSym`, que treballen conjuntament amb l'analitzador lèxic desenvolupat amb JFlex.

Mètode utilitzat: arbre explícit (AST)

En aquesta pràctica s'ha optat per un **arbre explícit**, és a dir, durant el procés de parsing **no només es reconeix la sintaxi**, sinó que **es construeixen explícitament objectes Java** que representen els diferents nodes de l'AST.

Cada regla sintàctica rellevant incorpora una **acció semàntica** (`{: ... :}`) que crea una instància d'un node concret del paquet `ast.*`. Aquest enfocament presenta diversos avantatges:

- Separa clarament **sintaxi concreta** (tokens, parèntesis, paraules clau) de la **sintaxi abstracta**.
- Facilita enormement l'anàlisi semàntica posterior (tipus, àmbits, verificacions).
- Permet una generació de codi més estructurada i modular.
- Fa el compilador més extensible i mantenible.

Per tant, el parser no retorna simples estructures intermèdies, sinó un **objecte `ProgramNode`** que representa l'arrel de l'AST complet del programa.

Precedència i associativitat

La gramàtica defineix explícitament la precedència dels operadors mitjançant directives `precedence`, resolent possibles conflictes *shift/reduce*. L'ordre establert reflecteix el comportament habitual dels llenguatges imperatius:

- Operadors lògics (`OR`, `AND`) amb precedència baixa.
- Operadors relacionals no associatius.
- Operadors aritmètics amb precedència més alta.
- Operadors unaris (`NOT`) amb associativitat a la dreta.

Aquesta definició garanteix que l'AST generat representi correctament l'estructura semàntica de les expressions sense necessitat d'afegir parèntesis artificials.

Gestió d'errors sintàctics

El parser redefineix els mètodes:

- `report_error`
- `syntax_error`
- `unrecovered_syntax_error`

per integrar-se amb l'**ErrorManager** del compilador. Quan es detecta un error sintàctic:

- Es registra el tipus d'error (**SINTACTIC**).
- S'indica la posició exacta (línia i columna).
- Es proporciona un missatge descriptiu.

A més, s'han inclòs produccions amb el símbol especial **error** per permetre una **recuperació parcial**, ignorant declaracions o instruccions incorrectes i continuant l'anàlisi quan és possible. Això millora notablement l'experiència d'ús i permet reportar múltiples errors en una sola passada.

Utilitat de l'AST en fases posteriors

L'AST construït pel parser és l'eix central del compilador. Gràcies a la seva estructura explícita:

- **Anàlisi semàntica**: es poden recórrer els nodes per comprovar tipus, àmbits, usos correctes d'identificadors i coherència de retorns.
- **Generació de codi intermedi o final**: cada node sap com traduir-se a la representació corresponent.
- **Depuració i extensibilitat**: l'AST facilita l'afegit de noves construccions al llenguatge sense reescriure el parser complet.

Anàlisi semàntica — SemanticAnalyzer.java

L'analitzador semàntic s'ha implementat seguint un **enfocament de traducció dirigida per la sintaxi (Syntax-Directed Translation, SDT)**. Això significa que, a partir de l'**AST construït pel parser**, es realitza un **recorregut sistemàtic de l'arbre** on, per a cada node, s'executen **regles semàntiques** que comproven la correcció del programa i propaguen informació semàntica cap als nodes superiors.

L'anàlisi no modifica l'estructura de l'AST, sinó que **l'anota**: s'hi afegixen tipus, modes d'ús, informació de retorn i marques d'error que seran essencials en les fases posteriors del compilador.

Estratègia general de funcionament

La classe `SemanticAnalyzer` actua com un **visitador manual** de l'AST. No s'utilitza el patró *Visitor* formal, sinó un conjunt de mètodes `gest_*` que corresponen directament als **tipus de node de l'arbre**.

El flux general és:

1. Es crida `analyze(ProgramNode)`.
2. Es comença pel node arrel (`gest_program`).
3. Cada `gest_node`:
 - Analitza el node actual.
 - Crida recursivament els `gest_*` dels nodes fills.
4. La informació semàntica (tipus, modes, errors) **es propaga cap amunt**.

Aquest esquema reflecteix fidelment el concepte de **traducció dirigida per la sintaxi**: la semàntica del programa es construeix a mesura que es recorre la seva estructura sintàctica.

Gestió per nivells de l'AST

- **`gest_program`**
 - Aquí es garanteix que tot el programa sigui analitzat en un **context global coherent** abans de tancar l'anàlisi.
- **`gest_decls / gest_decl`**
 - S'encarreguen de validar les declaracions de variables i constants:
- **`gest_type`**
 - Comprova que el tipus existeix i que realment és un tipus vàlid, associant-hi una descripció semàntica reutilitzable.
- **`novaTupla`**
 - Aquí es construeix la semàntica interna del tipus: camps, tipus de cada camp i estructura global.
- **`gest_method`**
 - Aquest mètode obre i tanca àmbits interns, permetent una correcta **anidació semàntica** i garantint que les regles d'àmbit i visibilitat es compleixin.
- **`gest_instr`**

Fa de **dispatcher semàntic** per a totes les instruccions del llenguatge. Segons el tipus concret del node. Aquesta separació permet definir regles semàntiques específiques per a cada construcció del llenguatge.

- **gest_cond_if / gest_cond_switch**

Verifiquen:

- Que les condicions siguin de tipus lògic o escalar (segons el cas).
- Que els blocs interns siguin semànticament correctes.
- La coherència entre expressions i casos (**switch**).

- **gest_loop_while / gest_loop_dowhile**

- Asseguren que les condicions dels bucles siguin booleanes i que el cos del bucle sigui analitzat dins el seu propi àmbit.

- **gest_ref**

És un dels punts clau de l'anàlisi semàntica. Determina:

- Si un identificador existeix.
- Quin tipus té.
- Quin és el seu mode d'ús (variable, constant, subprograma).
- Si l'ús és vàlid en el context actual.

- **gest_ref_camp**

Implementa la semàntica d'accés a camps de tuples:

- Comprova que la base és una tupla.
- Verifica que el camp existeix.
- Propaga el tipus del camp com a resultat de l'expressió.

Aquest mecanisme permet construir expressions complexes de manera recursiva i segura.

L'anàlisi d'expressions és el millor exemple de **traducció dirigida per la sintaxi**.

- **gest_expr**

Decideix com analitzar l'expressió segons la seva forma (binària, unària o terminal).

Un aspecte clau del disseny és la **propagació controlada d'errors**:

- Quan un node té error:
 - Es marca amb **hasError**.
 - Se li assigna un tipus **UNKNOWN**.
- Els nodes superiors detecten aquest estat i eviten errors en cascada.

Això permet:

- Informar de múltiples errors en una sola execució.
- Evitar missatges redundants o confusos.

- Mantenir l'anàlisi robusta fins i tot amb programes incorrectes.

Gestió de la taula de símbols

La taula de símbols del projecte s'ha plantejat com una **infraestructura semàntica centralitzada** que permet representar i consultar, de manera coherent i eficient, tota la informació associada als identificadors del programa: variables, constants, tipus definits, subprogrames, arguments i camps de tuples. El disseny prioritza dos objectius: **gestionar àmbits (scopes) amb visibilitat lexical** i **modelar la informació semàntica de forma tipada i extensible** mitjançant descripcions.

Organització per àmbits: entorn jeràrquic

El nucli del disseny és una **estructura jeràrquica d'àmbits**, on cada àmbit es representa amb una instància de `SymbolTable`. Aquesta instància conté:

- Un **diccionari local** `Map<String, Simbol>` amb els noms declarats dins aquell àmbit.
- Un enllaç al **pare** (`parent`) que apunta a l'àmbit exterior.

Aquesta organització implementa la visibilitat típica dels llenguatges imperatius:

- Les declaracions s'insereixen sempre en l'àmbit actual.
- Les consultes es resolen seguint la cadena `scope actual → pare → ... → global`, de manera que un identificador no trobat localment pot ser heretat d'un àmbit superior.

El resultat és un model que suporta de manera natural:

- **Blocs anidats** (funcions, procediments, `if/else`, bucles, `switch/case`).
- **Shadowing** (si es permet), ja que noms iguals en àmbits diferents no entren en conflicte mentre no coincideixin al mateix map local.
- Un comportament coherent amb l'AST: cada node que introdueix bloc pot activar un nou entorn.

Cada entrada de la taula de símbols s'emmagatzema com un objecte `Simbol`, que encapsula:

- El **nom** textual (**name**), que és la clau d'indexació.
- Una **descripció semàntica** (**Descripcio**) que indica quin rol té aquell nom i quines propietats semàntiques l'acompanyen.
- L'identificador d'àmbit (**scopeId**), útil per traça i depuració.

Descripcions: jerarquia semàntica extensible (**Descripcio**)

La classe abstracta **Descripcio** defineix un mecanisme uniforme per classificar símbols mitjançant un enumerat **TDesc** (variable, constant, procediment/funció, tipus, argument, camp). La idea és que **cada categoria d'identificador té requisits i regles pròpies**, i el compilador necessita distingir-les explícitament.

A la pràctica, això permet que el compilador pugui:

- Determinar si un identificador representa una **variable assignable**, una **constant**, un **subprograma**, o un **tipus**.
- Aplicar regles semàntiques específiques segons la categoria (p. ex., prohibir assignacions a constants, validar crides a subprogrames, resoldre tipus, etc.).

Aquest enfocament és el que fa que la taula de símbols sigui **escalable**: afegir una nova categoria o propietat no implica redissenyar el sistema, sinó ampliar la jerarquia de descripcions.

Tipus i predefinitos: “prelude” del llenguatge

L'àmbit global no comença buit: s'inicialitza amb un conjunt de símbols predefinitos que actuen com a **base del sistema de tipus** del llenguatge. Això inclou:

- Tipus primitius (enter, lògic, caràcter, cadena, void, etc.).
- Un tipus **unknown** per a **propagació d'errors**.
- Constants predefinides com **cert** i **fals** associades al tipus lògic.

Gestió d'errors

La gestió d'errors del compilador s'ha dissenyat com un **mecanisme centralitzat i transversal**, utilitzat per totes les fases del procés de compilació: anàlisi lèxica, sintàctica i

semàntica. L'objectiu principal no és aturar l'execució al primer error, sinó **detectar, acumular i presentar els errors de manera clara i ordenada**.

Totes les parts del compilador **informen dels errors a un únic gestor comú**, l'**ErrorManager**. Cada vegada que una fase detecta una situació incorrecta:

- crea un objecte **CompilerError**,
- i el registra mitjançant **ErrorManager.add(...)**.

D'aquesta manera, els errors no es perden ni es mostren de forma dispersa, sinó que queden **centralitzats** fins al final de la compilació.

Tipus d'errors

Cada error està classificat segons la fase on s'ha produït:

- **LÈXIC**: caràcters o símbols no reconeguts per l'analitzador lèxic.
- **SINTÀCTIC**: errors d'estructura del programa detectats pel parser.
- **SEMÀNTIC**: errors de significat, com tipus incompatibles, identificadors no declarats o usos incorrectes.

Aquesta classificació permet entendre ràpidament **en quina fase falla el programa** i facilita tant la depuració com la correcció per part de l'usuari.

Un error (**CompilerError**) conté:

- el **tipus** (lèxic, sintàctic o semàntic),
- la **línia i columna** on s'ha produït (si són conegudes),
- i un **missatge descriptiu**.

Gestió de memòria

La gestió de memòria del compilador segueix un **model clàssic basat en pila**, adequat per a un llenguatge que **permet funcions i procediments anidats** i, per tant, necessita respectar la **visibilitat lèxica** dels identificadors en temps d'execució.

Estructura general de memòria

El programa s'organitza conceptualment en tres zones principals:

- **Zona de codi**, on resideixen les instruccions executables.
- **Zona de dades**, que conté literals (especialment cadenes), buffers i estructures auxiliars com el **vector de display**.
- **Pila d'execució**, utilitzada per gestionar crides a subprogrames mitjançant **blocs d'activació**.

Aquest esquema permet una execució ordenada de subprogrames i una gestió clara dels àmbits.

Blocs d'activació i crides

Cada crida a una funció o procediment crea un **bloc d'activació** a la pila. Aquest bloc emmagatzema:

- la informació necessària per tornar al cridant,
- l'estat del procediment anterior,
- i l'espai reservat per a variables locals i temporals.

En entrar a un subprograma s'executa un **pròleg**, que crea el nou bloc d'activació i prepara l'entorn d'execució. En sortir, un **epíleg** desmunta aquest bloc i restaura l'estat anterior, garantint que la pila quedi consistent.

Vector de display (DISP)

Per donar suport a **subprogrames anidats**, el compilador utilitza un **vector de display**. El display és una taula global de punters on cada entrada apunta al bloc d'activació actiu d'un determinat **nivell lèxic**.

La seva funció principal és permetre l'**accés directe a variables no locals**, és a dir, declarades en procediments externs. En lloc de recórrer cadenes d'enllaços entre blocs, el display permet obtenir immediatament el marc correcte, fent l'accés més simple i eficient.

Quan s'entra en un procediment:

- s'actualitza la posició corresponent del display amb el nou bloc d'activació.

Quan se'n surt:

- es restaura el valor anterior del display.

Accés a variables i retorns

- Les **variables locals i temporals** s'accedeixen mitjançant desplaçaments respecte al registre base del bloc d'activació actual.
- Les **variables no locals** s'accedeixen primer a través del display i després aplicant el desplaçament corresponent.
- Els **paràmetres** es passen per pila, i el **valor de retorn** d'una funció es comunica mitjançant un registre dedicat.

Generació de codi

Optimitzacions

El compilador incorpora una fase d'optimització sobre el **codi intermedi** (llista d'**Instruction**) abans de generar l'assemblador. Es tracta d'optimitzacions **locals** (**peephole**), aplicades mitjançant un recorregut seqüencial del codi, amb l'objectiu principal de **reduir salts innecessaris** i **simplificar temporals** sense alterar la semàntica del programa.

Optimitzacions implementades

- **Simplificació de brancaments adjacents (inversió de condició)**

Detecta el patró típic:

```
IF cond -> L1 ; GOTO L2 ; SKIP L1
```

i el transforma en:

```
IF !cond -> L2 ; SKIP L1
```

Això elimina un salt incondicional i fa el flux més compacte.

- **Jump threading (redirecció de salts)**

Quan un **GOTO** apunta a una etiqueta que immediatament condueix a un altre **GOTO**, es redirecciona el primer salt cap al destí final. Això evita "salts en cadena" i simplifica el control de flux.

- **Eliminació de salts trivials**

Si apareix el patró:

```
GOTO L ; SKIP L
```

el salt és redundant i s'elimina, ja que el control cauria directament a l'etiqueta.

- **Eliminació de codi mort després d'un **GOTO****

Després d'un salt incondicional, les instruccions següents no són executables fins a arribar a una etiqueta (**SKIP**). Aquesta optimització descarta aquest segment mort, reduint codi sense efecte.

- **Propagació de literals**

Si un temporal rep un literal i immediatament es copia a una variable:

```
t = literal ; x = t
```

es transforma en:

```
x = literal
```


i, si el temporal no s'usa més dins el mateix procediment, s'elimina també l'assignació original.

- **Propagació de còpies (copy propagation)**

En el cas:

$t = x ; y = t$

es transforma en:

$y = x$

i, igual que abans, la primera còpia es pot eliminar si el temporal no torna a ser utilitzat.

Per evitar canvis semàntics, les eliminacions de temporals només es fan quan es comprova que el temporal **no es reutilitza més endavant dins el mateix procediment**. Per això, l'optimitzador limita l'anàlisi fins a trobar un canvi de context (inici o final de procediment) i ignora usos que no són realment operands (per exemple, offsets d'indirecció).

Generació de codi assemblador

La generació de codi final transforma el codi intermedi del compilador (llista d'**Instruction**) en un fitxer d'**assemblador per a l'arquitectura Motorola 68000 (X68)**. No s'han incorporat llibreries externes pròpies: la interacció amb entrada/sortida es resol mitjançant els **serveis del simulador** (trucades **TRAP #15**).

Arquitectura escollida i convencions bàsiques

S'ha treballat amb un model **32 bits** propi del 68000 (operacions **.L**), distingint:

- **Registres de dades (D*)** per a càlculs i comparacions.
- **Registres d'adreça (A*)** per a punters i adreçament indirecte.

Concretament, el generador fixa alguns registres amb un rol clar:

- **A6** com a *base pointer* del procediment actual (accés a locals, temporals i paràmetres via desplaçaments).
- **A0** com a registre auxiliar per carregar el **DISP** quan cal accedir a variables no locals.

A nivell d'organització del fitxer, es generen dues seccions:

- **TEXT** (codi): punt d'entrada **START** i etiquetes **E...** per salts i procediments.
- **DATA** (dades): literals, buffers i estructures auxiliars.

Traducció general de les instruccions

El generador recorre seqüencialment la llista d'instruccions del codi intermedi i aplica una traducció directa:

- **Flux de control**
 - **SKIP** crea etiquetes (**E<id>:**).
 - **GOTO** es tradueix a un salt incondicional (**BRA.W**).
 - Les condicions (**IF_***) es tradueixen amb el patró: carregar operands → **CMP** → salt condicional (**BEQ, BNE, BGE**, etc.).
- **Operacions aritmètiques i lògiques**
 - Es carreguen operands a registres (**D0, D1**), s'aplica l'operació (**ADD, SUB, MULS, DIVS, AND, OR**) i es desa el resultat a la destinació.
- **Còpies i assignacions**
 - Per a valors escalars, s'utilitza una còpia directa (**MOVE.L**).
 - Per a literals, el generador emet la càrrega immediata (**#literal**) cap al destí.

Per suportar accés indirecte (p. ex. camps d'una tupla), el codi intermedi inclou instruccions específiques:

- **Lectura indirecta (**IND_VAL**)**: calcula l'adreça base i llegeix amb un desplaçament (offset) per obtenir el camp.
- **Escriptura indirecta (**IND_ASS**)**: calcula base + offset i escriu el valor.

Gestió de cadenes (**cadena**)

Les cadenes es gestionen com a **punters a buffers**:

- A la secció de dades:
 - els literals de cadena es declaren com bytes acabats en 0 (**DC.B ... ,0**),
 - les variables **cadena** tenen un buffer reservat (**DS.B 512**).

Com que una cadena no és un escalar, les operacions principals es resolen amb subrutines pròpies:

- **COPY_STRING**: copia byte a byte fins al terminador.
- **APPEND_STRING**: cerca el final i concatena.

La concatenació (**CONCAT**) reutilitza aquestes rutines, i quan el destí és un temporal es fa servir un **buffer temporal** (BTS) per evitar reservar memòria dinàmica.

Entrada/sortida i dependències externes

No s'empren llibreries externes del projecte. L'entrada/sortida es resol amb el mecanisme del simulador mitjançant **TRAP #15**, amb codis diferents segons el tipus:

- lectura/escriptura d'enters i caràcters amb serveis dedicats,
- escriptura de cadenes passant el punter a buffer,
- booleans representats com **-1/0** i, per mostrar-los, convertits a les cadenes literals **"cert"** i **"fals"**.

Execució i validació

Llibreries necessàries

Per generar i executar el compilador s'utilitzen tres dependències externes:

- **JFlex (`jflex-full-1.9.1.jar`)**
Eina de generació de l'analitzador lèxic. A partir del fitxer `.flex` genera la classe `AnaLex`, que s'encarrega de reconèixer tokens i retornar-los al parser.
- **Java CUP (`java-cup-11b.jar`)**
Eina de generació de l'analitzador sintàctic. A partir del fitxer `.cup` genera:
 - el parser (`Parser.java`),
 - i l'enumeració de terminals (`ParserSym.java`).
- **Java CUP Runtime (`java-cup-11b-runtime.jar`)**
Llibreria necessària en temps d'execució, ja que el compilador fa servir classes de CUP com `java_cup.runtime.Symbol` i el mecanisme de comunicació lexer–parser.

Aquestes llibreries s'empren per construir el front-end (lèxic+sintàctic) i per executar-lo.

Procediment d'execució

El projecte s'ha desenvolupat i provat principalment amb **IntelliJ IDEA**, que s'encarrega de compilar el codi Java i generar els fitxers `.class` dins la carpeta de sortida de l'IDE. En aquest projecte, IntelliJ genera les classes compilades a:

- `out/production/compilador/`

A més, el compilador depèn en temps d'execució de **Java CUP Runtime** (`java-cup-11b-runtime.jar`), ja que el parser i el mecanisme lexer–parser utilitzen classes del paquet `java_cup.runtime`.

El codi s'ha compilat amb una **versió moderna de Java** (en el desenvolupament s'ha treballat amb una JDK recent, p. ex. **Java 23/25**). Per executar-lo des de línia de comandes, el runtime utilitzat ha de ser **compatible amb el bytecode** generat (és a dir, convé utilitzar la mateixa JDK o una versió igual o superior a la de compilació).

Execució des d'IntelliJ (o IDE alternatiu)

1. Obrir el projecte amb IntelliJ.
2. Compilar (automàticament en fer "Run" o bé amb "Build Project").
3. Executar la classe `compiler.Main` des de l'IDE, passant arguments si cal:
 - Argument 1: fitxer d'entrada (codi font a compilar)
 - Argument 2 (opcional): directori de sortida

Aquesta opció és la més còmoda durant el desenvolupament perquè IntelliJ configura internament el classpath del projecte.

Execució des de terminal

Per executar el compilador des de terminal **sense modificar el codi**, s'utilitzen les classes ja compilades que IntelliJ ha generat a `out/production/compilador` i s'afegeix el runtime de CUP al classpath.

Exemple d'execució:

```
java -cp "out\production\compilador;java-cup-11b-runtime.jar"
compiler.Main programa.txt out
```

On:

- `out\production\compilador` conté les classes `.class` compilades (incloent `compiler.Main`).
- `java-cup-11b-runtime.jar` és imprescindible perquè el parser funcioni en execució.
- `programa.txt` és el fitxer font a compilar.
- `out` és el directori de sortida (si no s'indica, el `Main` utilitza `out/` per defecte).

Aquesta forma d'execució permet compilar **qualsevol fitxer d'entrada** simplement canviant `args[0]`, sense haver de modificar ni el nom ni el contingut del compilador.

En executar `compiler.Main`, el compilador processa el fitxer d'entrada i, si no hi ha errors, completa tota la cadena de compilació (front-end + back-end). El resultat principal de la compilació és la generació de **dos fitxers assemblador** per a 68k:

- **program_no_opt.X68**: codi assemblador generat a partir del TAC **sense** passar per la fase d'optimització.
- **program_opt.X68**: codi assemblador generat a partir del TAC **optimitzat** pel mòdul **optimizer.Optimizer**.

Aquests dos fitxers permeten comparar directament l'efecte de les optimitzacions aplicades (mateixa entrada, diferent assemblador generat).

La sortida s'escriu dins el directori indicat pel segon argument:

- Si es passa **args[1]**, s'utilitza com a carpeta de sortida.
- Si no es passa, el compilador utilitza **out/** per defecte.

Execució del **.X68** amb **EASY68K**

Els fitxers generats (**.X68**) són programes **assemblador Motorola 68000** pensats per ser carregats en un IDE/simulador com **EASY68K**. El compilador no executa el programa final; únicament genera el codi assemblador.

El flux és:

1. **Obrir EASY68K.**
2. **Carregar** el fitxer generat (**program_no_opt.X68** o **program_opt.X68**).
3. **Assemble** (compilar assemblador) dins EASY68K.
4. **Run/Execute** per simular l'execució del programa 68k.

Casos de prova

Els casos de prova han estat executats en el compilador i s'han desat els resultats generats per el compilador, tant de sortida com de codi. Aquests s'han emmagatzemat dins la ruta `tests/`, on cada cas de prova té el seu fitxer `.txt` corresponent i un directori `/out/` que conté les sortides produïdes. En els casos correctes la sortida són tots els elements que ha usat el compilador per generar els fitxers `.X68` (tant optimitzat com sense optimitzar) i en els casos erronis la sortida és el propi fitxer d'errors generat per el compilador, anomenat `errors.txt`.

Cas de prova 1

Aquest cas de prova verifica el funcionament correcte dels elements bàsics del llenguatge, incloent la declaració de variables globals de tipus `enter`, `cadena` i `logic`, les operacions d'entrada i sortida, l'avaluació d'expressions relacionals i l'assignació de valors booleans. El programa llegeix dades introduïdes per l'usuari, construeix una expressió lògica a partir d'una comparació i mostra informació per pantalla, comprovant així la correcta integració entre expressions, assignacions i I/O.

A més, el cas posa a prova la definició i crida de procediments amb pas de paràmetres per valor, així com l'ús d'estructures condicionals completes (`si - sino - fsi`) dins un subprograma. D'aquesta manera es valida la gestió d'àmbits, el control de flux i la comunicació entre el bloc principal i els subprogrames, demostrant que el compilador executa correctament un programa estructurat sense errors.

```
principal prova01:
  enter edat
  cadena nom
  logic major

  procediment mostra_majoria(esMajor: logic):
    inici
      si (esMajor) llavors
        sortida("Ets major d'edat.")
      sino
        sortida("Ets menor d'edat.")
      fsi
    fi
  fiprocediment
```

```

inici
    sortida("Introdueix el teu nom:")
    entrada(nom)

    sortida("Introdueix la teva edat:")
    entrada(edat)

    major := (edat >= 18)

    sortida("Nom: " + nom)
    mostra_majoria(major)
fi
fprincipal

```

Cas de prova 2

Aquest cas de prova verifica l'ús correcte dels **tipus compostos mitjançant tuples**, combinant-los amb funcions, expressions lògiques complexes i estructures de selecció múltiple. El programa defineix una tupla que agrupa diferents tipus bàsics (**cadena**, **enter**, **logic** i **caracter**) per modelar la informació d'un sensor, i comprova l'accés i l'assignació als seus camps mitjançant la notació amb punt.

A més, el cas posa a prova la **definició d'una funció amb variables locals i valor de retorn**, l'avaluació de **condicions compostes amb els operadors lògics i i o**, i l'ús de la instrucció **cas** per seleccionar el comportament del programa en funció del valor retornat per la funció. D'aquesta manera es valida la gestió d'àmbits, el control de flux i la integració entre tipus compostos, funcions i estructures de decisió dins el llenguatge.

```

principal prova02:
    tupla {nom: cadena, temperatura: enter, actiu: logic, codi: caracter}
    Sensor

    Sensor s
    enter nivell_alerta

    funcio enter calcula_alerta(temp: enter, actiu: logic):
        inici
            enter codi

```



```

        si ((temp > 70 i actiu) o (temp > 90)) llavors
            codi := 2
        sino
            codi := 1
        fsi
    fi
    tornar codi
fifuncio

inici
    s.nom := "Sensor Central"
    s.temperatura := 70
    s.actiu := cert
    s.codi := 'S'

    nivell_alerta := calcula_alerta(s.temperatura, s.actiu)
    cadena nomS := s.nom

    cas nivell_alerta:
        1:
            sortida("Estat normal del sensor " + nomS)
        2:
            sortida("ALERTA: temperatura elevada del sensor " + nomS)
        altre:
            sortida("Estat desconegut")
    fcas
fi
fprincipal

```

Cas de prova 3

Aquest cas de prova verifica el funcionament dels **bucles d'iteració** i del **control de flux** mitjançant un algorisme clàssic: la comprovació de si un nombre és primer. El programa utilitza un bucle **mentre** amb una condició composta que combina comparacions aritmètiques i una variable lògica de control, així com condicions anidades dins el bucle per detectar divisors del nombre introduït.

A més, el cas posa a prova l'ús d'**expressions aritmètiques**, **comparacions d'igualtat i desigualtat** (**=**, **!=**, **<=**), l'actualització de variables dins un bucle i la integració d'**entrada i sortida** amb el procés de càlcul. D'aquesta manera es valida que el compilador és capaç d'executar correctament algorismes iteratius típics, gestionant correctament les condicions, els bucles i l'estat del programa.

```

principal prova03:
  enter n
  enter x
  logic es_primer

  inici
    sortida("Introdueix un nombre enter major que 1:")
    entrada(n)

    es_primer := cert
    x := 2

    enter resultat

    mentre (x <= n / 2 i es_primer) fer
      resultat := (n / x) * x

      si (resultat == n) llavors
        es_primer := fals
      sino
        x := x + 1
      fsi
    fmentre

    si (es_primer i n != 1) llavors
      sortida("El nombre es primer")
    sino
      sortida("El nombre no es primer")
    fsi
  fi
fprincipal

```

Cas de prova erroni 1

Aquest cas erroni està pensat per validar que el compilador detecta de manera robusta **errors semàntics bàsics** relacionats amb la taula de símbols i el sistema de tipus. En primer lloc, es declara dues vegades la variable **x** dins el mateix àmbit, provocant un error de **redefinició d'identificador** (línia 3). Tot seguit, s'utilitza la variable **y** sense haver estat declarada prèviament, generant un error d'**identificador no declarat** (línia 6). Aquests dos punts comproven que el compilador gestiona correctament l'alta d'entrades a la taula de símbols i la verificació d'usos de noms.

A continuació, el programa força errors de **compatibilitat de tipus** i d'ús correcte d'estructures de control. L'assignació `x := "hola"` intenta guardar una **CADENA** dins un **ENTER**, cosa que el comprovador de tipus rebutja (línia 8). Igualment, el `si ("cert") llavors` és incorrecte perquè la condició d'un `if` ha de ser de tipus booleà (**logic**) i aquí s'està proporcionant una **CADENA** (línia 10). Finalment, es fa una crida a `mostra_majoria(x)` sense que aquest subprograma estigui declarat, generant un error de **crida a subprograma inexistent** (línia 14). En conjunt, el programa falla perquè incompleix regles semàntiques fonamentals: unicitat de declaracions, ús previ a la declaració, coherència de tipus i resolució correcta de crides a subprogrames.

```
principal err01:
  enter x
  enter x      ## (1) Identificador duplicat al mateix àmbit

  inici
    y := 3     ## (2) Variable no declarada (y)

    x := "hola" ## (3) Assignació incompatible: enter := cadena

    si ("cert") llavors ## (4) Condició no és logic (és cadena)
      sortida("dins si")
    fsi

    mostra_majoria(x) ## (5) Crida a procediment/funció no declarat
  fi
fprincipal
```

Cas de prova erroni 2

Aquest cas erroni està orientat a comprovar la **detecció d'errors semàntics** relacionats amb **tipus** i **ús correcte de subprogrames** (funcions i procediments). Dins la funció `suma`, es força una assignació incompatible quan s'intenta guardar un **CARACTER** a la variable `t` declarada com a **CADENA** (línia 17). Tot seguit, el programa també provoca un error de coherència de retorn: la funció està declarada com a **funcio enter**, però es retorna `t`, que és de tipus **CADENA** (línia 20). Aquests dos errors validen que el compilador comprova

tant la compatibilitat de tipus en assignacions locals com el tipus del valor retornat per una funció.

Al bloc principal es generen errors semàntics addicionals que simulen equivocacions habituals d'usuari amb crides a subprogrames. Es prova que un **procediment no pot usar-se com a expressió** intentant assignar el resultat d'`imprimeix("Hola")` a `res` (línia 24). També es verifica la **compatibilitat de tipus en assignacions** (`text := suma(3, 4)` assigna un `ENTER` a una `CADENA`, línia 26), el control del **nombre d'arguments** a una crida de funció (`suma(10)` amb un sol paràmetre, línia 28) i la validació del **tipus dels paràmetres** en una crida a procediment (`imprimeix(5)` rep un `ENTER` quan s'esperava `CADENA`, línia 30). En conjunt, el programa falla perquè vulnera múltiples regles semàntiques del llenguatge: tipatge estàtic, firma de subprogrames i distinció entre procediments (sense retorn) i funcions (amb retorn).

```
principal err02:
  enter res
  cadena text

  procediment imprimeix(m: cadena):
    inici
      sortida(m)
    fi
  fiprocediment

  funcio enter suma(a: enter, b: enter):
    inici
      cadena t
      si (a >= b) llavors
        t := "0"
      sino
        t := 'A' ## (1) Assignació de tipus incorrecte: cadena <-
caràcter
      fsi
    fi
    tornar t ## (2) Retorn de tipus incorrecte: enter <- cadena
  fifuncio

  inici
    res := imprimeix("Hola")    ## (3) Intent d'assignar el resultat
d'un procediment (no retorna)
```

```

        text := suma(3, 4)          ## (4) Assignació incompatible:
cadena <- enter

        res := suma(10)            ## (5) Nombre de paràmetres
incorrecte a la funció

        imprimeix(5)              ## (6) Tipus de paràmetre
incorrecte: espera cadena, rep enter
    fi
fprincipal

```

Cas de prova erroni 3

Aquest cas erroni està dissenyat per provocar principalment **errors lèxics** i diversos **errors sintàctics derivats** durant l'anàlisi. En primer lloc, a la línia `x := x @@ y` apareixen **dos símbols @ desconeguts** pel llenguatge, fet que genera dos errors lèxics consecutius.

A partir d'aquests tokens d'error, el parser no pot seguir l'estructura esperada de les instruccions i reporta errors sintàctics com ara `token inesperat 'ERROR'` i “instrucció ignorada”, que són mecanismes de recuperació. També apareix un error sintàctic específic a `si () llavors`, ja que la condició és buida: el tancament de parèntesi (`TANC_PAR`) arriba quan encara s'esperava una expressió vàlida. Finalment, l'ús de `}` dins la cadena (`TANC_CLAU`) també trenca la forma correcta de la crida a `sortida`, i el parser acaba ignorant la instrucció. En conjunt, el programa falla perquè la fase lèxica genera tokens invàlids i, tot seguit, la fase sintàctica no pot construir una seqüència d'instruccions correcta a causa d'expressions mal formades i tokens inesperats.

```

principal err03:
    enter x
    enter y

    inici
        x := 1
        y := 2

        x := x @@ y

```

```
    si () llavors
        sortida("Fi de prova}
    fi
fprincipal
```

Conclusions

La realització d'aquest compilador ha estat un **repte personal i acadèmic de gran envergadura**, tant per la quantitat de temps invertit com per la complexitat dels conceptes implicats. Ha estat, sens dubte, una de les pràctiques que més dedicació ha requerit al llarg de la carrera, ja que no es tracta només d'escriure codi, sinó d'entendre i coordinar correctament totes les fases que intervenen en el procés de compilació. Tot i l'esforç que ha suposat, el moment en què el compilador comença a funcionar de manera estable i és capaç d'analitzar i executar programes complets resulta especialment satisfactori i compensa àmpliament el treball realitzat.

Aquest projecte ha permès abordar una implementació que **no s'havia treballat mai amb aquest nivell de profunditat** durant el grau. Fins ara, la major part de les assignatures s'havien centrat en l'ús de llenguatges com Java, C, Python, Ada o Go per desenvolupar aplicacions, però sense entrar realment en què passa "per sota" quan un programa és traduït i executat per la màquina. El desenvolupament del compilador ha posat en relleu la **complexitat real que amaguen els llenguatges de programació**, així com la importància de cada fase (lèxica, sintàctica i semàntica) per garantir que el codi de l'usuari sigui correcte i executable.

Durant el procés ha estat necessari **cercar, estudiar i assimilar documentació de JFlex i CUP**, dues eines que no s'havien utilitzat prèviament al llarg de la carrera. L'aprenentatge del seu funcionament i de les seves sintaxis específiques ha estat un repte afegit, ja que no segueixen el mateix paradigma que els llenguatges de programació tradicionals. Tot i això, aquesta dificultat ha contribuït a entendre millor com es construeixen els analitzadors lèxics i sintàctics que utilitzen molts compiladors reals.

A més, es tracta d'un **projecte ampli i altament modular**, on intervenen nombroses classes i components que han de comunicar-se de manera coherent. La integració entre les diferents parts del compilador, així com la gestió dels fluxos d'informació entre les fases d'anàlisi, ha requerit una planificació acurada i una visió global del sistema. Aquest aspecte ha reforçat la importància del disseny i de l'organització del codi en projectes de certa complexitat.

Al llarg del desenvolupament també han sorgit errors que han requerit **dies d'anàlisi i depuració** per identificar-ne la causa. En molts casos, aquests problemes estaven provocats per petits descuits o detalls aparentment simples, però han resultat molt útils des

del punt de vista formatiu. Aquest procés ha ajudat a adquirir una major capacitat d'anàlisi, paciència i rigor, i ha contribuït a evitar errors similars en desenvolupaments posteriors.

Finalment, tot i que és poc probable haver d'implementar un compilador complet en l'àmbit professional, ja que la majoria dels compiladors utilitzats actualment ja estan àmpliament desenvolupats i optimitzats, aquest projecte ha proporcionat un **aprenentatge molt valuós**. Permet entendre el funcionament intern dels llenguatges de programació, valorar la complexitat dels sistemes que els fan possibles i adquirir una visió més profunda i crítica sobre una de les peces fonamentals de la informàtica modern.